B.Sc. (Data Science)

DS101T : Problem Solving and Python Programming

Unit 2: Introduction to Python

Mrs. Reshma Masurekar
Dr. D. Y. Patil Arts, Commerce and Science College,
Pimpri, Pune 18

1.2 Conditional statements-If, If-Else, nested if-else, Examples.

1.3 Looping-For, While, Nested loops, Examples

1.4 Control Statements-Break, Continue, Pass.

Conditional statements-If, If-Else, nested if-else

- is a conditional statement.
- It is used to execute a block of code only when a specific condition is met.

Statement	Description
If Statement	The if statement is used to test a specific condition. If the condition is true, a block of code (if-block) will be executed.
If - else Statement	It is similar to if statement except the fact that, it also provides the block of the code for the false case of the condition to be checked. If the condition provided in the if statement is false, then the else statement will be executed.
if…elif…else	
Nested if Statement	Nested if statements enable us to use if ? else statement inside an outer if statement.

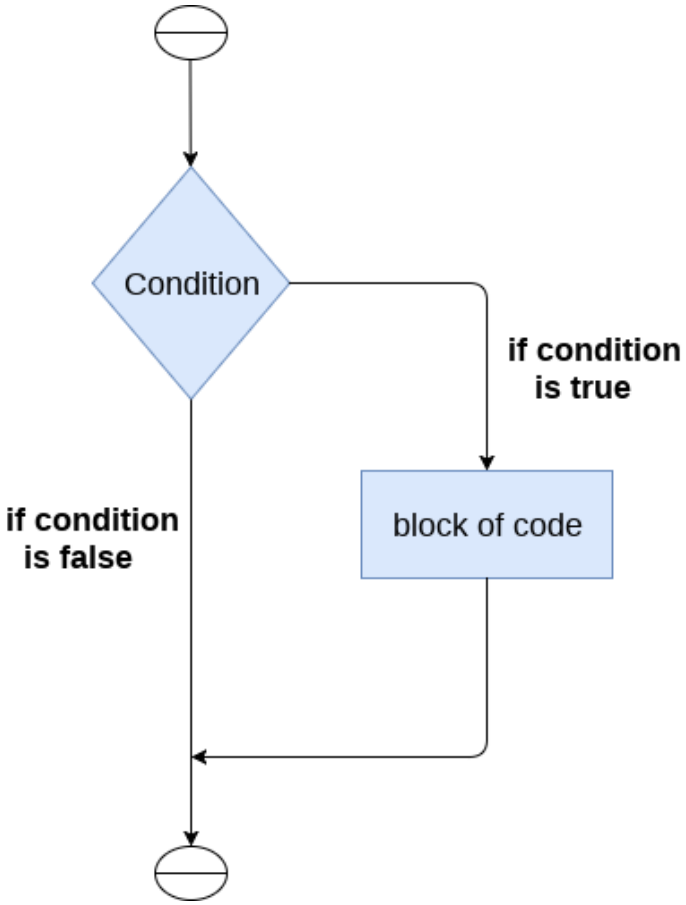
The if statement

The if statement is used to test a particular condition and if the condition is true, it executes a block of code known as if-block.

Syntax:

```
if condition:
        # body of if statement
```

- If condition is True, the body of the if statement is executed.
- If condition is False, the body of the if statement will be skipped from execution.



if...else Statement

If the condition is true, then the if-block is executed. Otherwise, the else-block is executed.

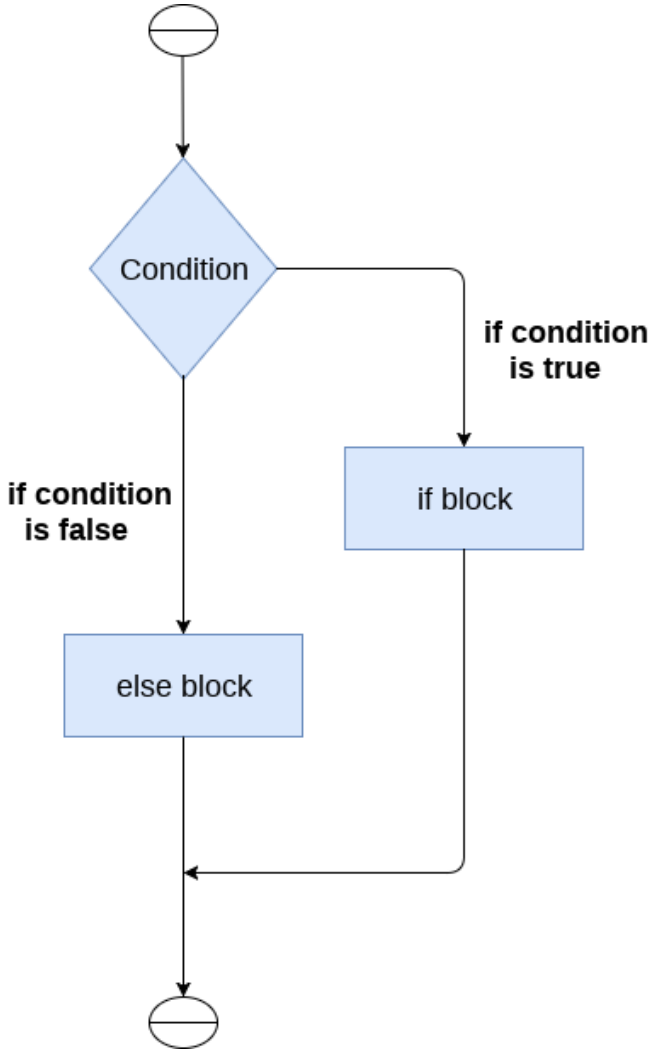
Syntax:

```
if condition:
        # body of if statement
else:
        # body of else statement
```

if statement evaluates to

True - the body of if executes, and the body of else is skipped.

False - the body of else executes, and the body of if is skipped

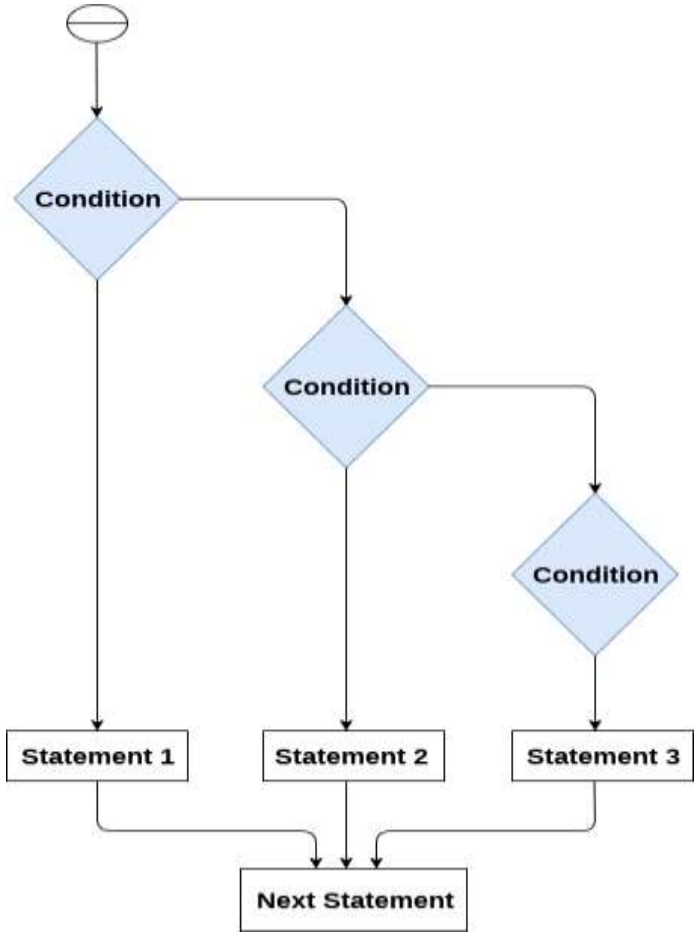


if…elif…else Statement

- To make a choice between more than two alternatives, the if…elif…else statement is used.

Syntax:

```
if condition1:
        # code block 1
elif condition2:
        # code block 2
else:
        # code block 3
```



Nested if

- if statement inside existing if statements, is called *nested* if statements.
- nested **if statrement is used in** a situation when user want to check for additional conditions after the initial one resolves to true.
- **Syntax:**

```
if boolean_expression1:
    statement(s)
    if boolean_expression2:
        statement(s)
```

Example1: check if number is greater than 0

```
number = int(input('Enter a number: '))
if number > 0:
    print(f'{number} is a positive number.')
print('A statement outside the if statement.')
```

Example2: Python Program to check is the candidate is Eligible to Vote

```
age = int (input("Enter your age: "))
if age>=18:
    print("You are eligible to vote !!");
else:
    print("Sorry! you have to wait !!");
```

Example3: check if number is greater than 0

```
number = int(input('Enter a number: '))
if number > 0:
    print('Positive number')
elif number < 0:
    print('Negative number')
else:
    print('Zero')
print('This statement is always executed')
```

Python *for* loop

- A for loop is used to iterate over a sequence (a list, a tuple, a dictionary, a set, or a string).
- For loop is used when user know exactly how many times to execute a loop.
- **Syntax:**

  for iterator_var in sequence:
  ```
      statements(s)                    #Body of for loop
  ```
- **Syntax:**

  Subjects=[“C”,”C++”,”Java”,”Python”]

  for i in Subjects:
  ```
      print(i)
  ```

```
for i in range(5):
    print(i)

for i in range(1,5):
    print(i)

print("range with increament...")
for i in range(1,10,2):
    print(i)

print("\nString Iteration")
str = "Data Science"
for i in str:
    print(i)

print("List Iteration")
list1=["Cw","C++","Java","Python"]
for i in list1:
    print(i)
```

```
print("\nTuple Iteration")
tuple1 = ("Cw","C++","Java","Python")
for i in tuple1:
    print(i)

print("\nSet Iteration")
set1 = {1, 2, 3, 4, 5}
for i in set1:
    print(i)

print("\nDictionary Iteration")
dict1={101:"Python",102:"Stat",103:"Maths“}
for i in dict1:
    print(i,dict1[i])
```

Nested for loop

- A nested loop is a loop inside a loop.
- The "inner loop" will be executed one time for each iteration of the "outer loop“.

- **Syntax:**

```
for iterator_var 1 in  sequence1:
        for iterator_var 2 in  sequence2:
                statements(s)
```

- **Example:**

```
for i in range(2):
        for j in range(11,13):
                print(i,j)
```

Output:

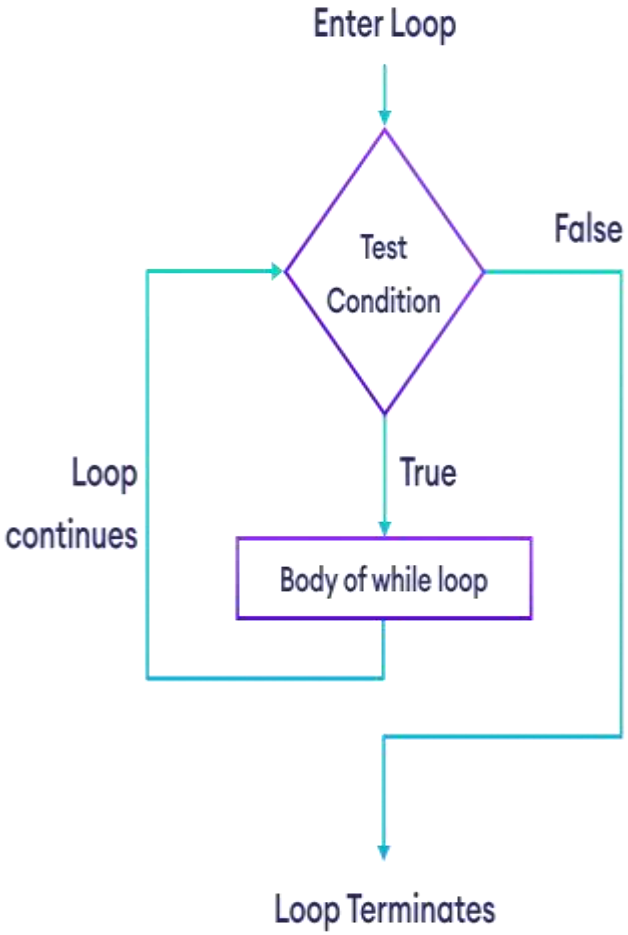
```
0 11
0 12
1 11
1 12
```

while loop

- **Python While Loop** is used to execute a block / set of statements repeatedly until a given condition is satisfied or True. When the condition becomes false, the line immediately after the loop in the program is executed.
- while loop is used when user dont know exactly how many times to execute a loop beforehand.
- **Syntax:**

  while condition:

  statement(s) #Body of while loop



```
str="Welcome to Data Science"
i=1
while i<=5:
    print(str)
    i=i+1
```

```
i=1
while i<=5:
    print(i)
    i=i+1
```

```
i=1
while i<=50:
    if i%5==0:
        print(i)
    i=i+1
```

break, continue, pass

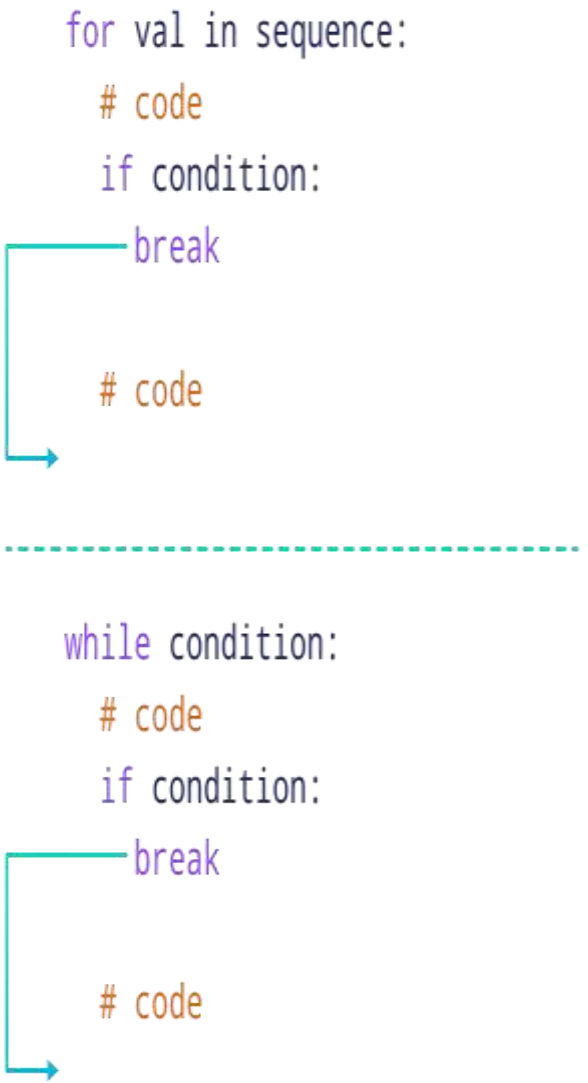
- break, continue, pass are control flow statements.
- **break** exits the loop entirely when a specific condition is met
- **continue** skips the current iteration and proceeds to the next one
- **pass** for loops cannot be empty, but if you for some reason have a for loop with no content, put in the pass statement to avoid getting an error.

break

- **break** exits the loop entirely when a specific condition is met.
- break statement terminates the loop immediately when a specific condition is met.
- **Syntax:**

  break

- **Note:** The break statement is usually used inside decision-making statements such as if...else.



continue

- The continue statement skips the current iteration of the loop when the specific condition is met and continue/jump with the next iteration of the loop.
- **Syntax:**

  continue

```
for val in sequence:
    # code
    if condition:
        continue

    # code
```

```
while condition:
    # code
    if condition:
        continue

    # code
```

pass

- No-op statement or null statement in Python.
- It is used to write empty loops. (It does nothing when executed)
- Pass is also used for empty control statements, functions, and classes.
- **Syntax:**

```
pass
```

#demo of break statement

```
for i in range(5):
   if i==3:
      break
   print(i)
```

Output:

0
1
2

 # demo of continue statement

```
i=1
for i in range(5):
   if i==3:
     continue
   print(i)
```

Output:

0
1
2
4

 # demo of pass statement

```
for i in range(5):
   pass
print("Outside for loop...")
```

Output: Outside for loop...